PRISM
PREPARING AND INSPIRING STUDENT MINDS

Pulse

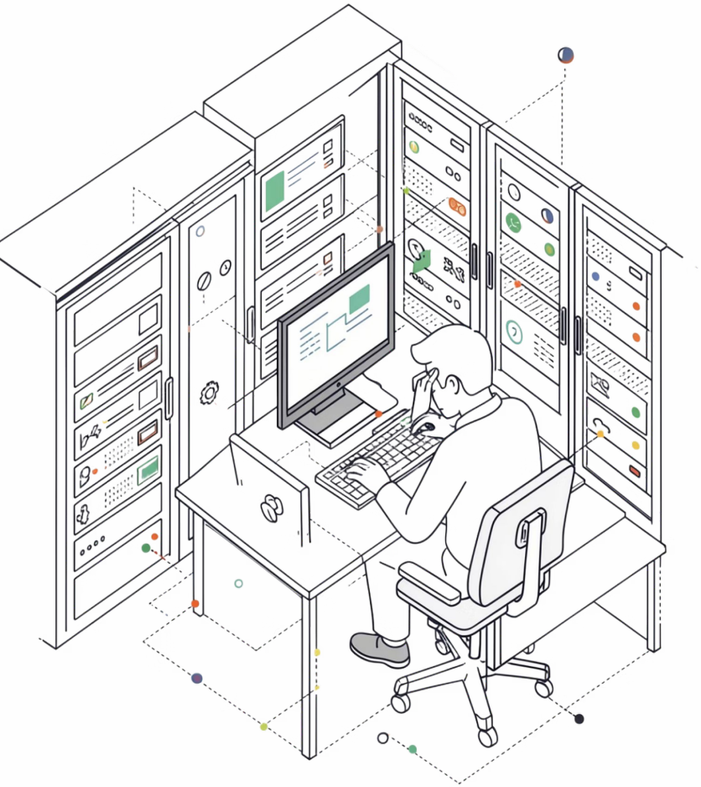
The Zero-Latency Execution Firewall for Terminals.

Theme 3: Productivity Platforms

By Arya Wadhwa, Dilraj Singh

2nd Year, CSE-AIML at RV College of Engineering

The Invisible Compliance Gap



Theme 3: Productivity Platforms

AI helps developers *write* code faster, but AI security tools are usually too slow — high latency — to protect them when *executing* commands on live servers. The gap between writing and running is where disasters happen.

The Real Problem

Risk Reduction

Eliminates the deployment anxiety that paralyzes engineers.

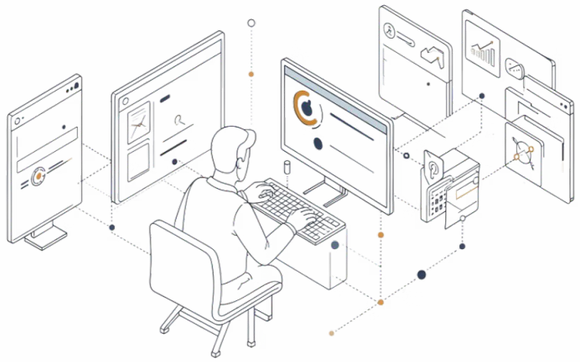
Blind Execution

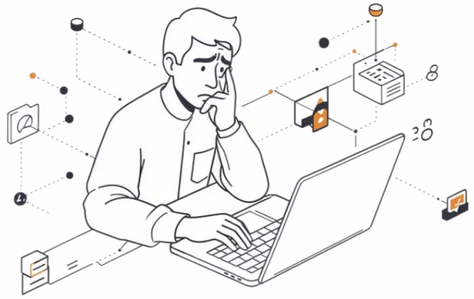
Terminals execute intent immediately. There is no "preview" for system changes — what you type, happens.

The Latency Flaw

Waiting 5 seconds for a cloud LLM to approve a terminal command completely destroys developer UX and adoption.

Target Audience & Their Pain





> The Ops Engineer: As a DevOps engineer, I want to see a git-like diff of my terminal commands before they execute, so that I can safely make emergency changes at 3 AM without waking up my manager for approval.
>
> The Compliance Lead: As an auditor, I want automatic, structured logs of file-system changes, so that I don't have to manually parse unreadable .bash_history files for SOC2 evidence.

Target User

Sysadmins, DevOps Engineers, and SREs managing production environments — people whose single keystroke can bring down a service.

Their Challenges

- Constant anxiety over destructive commands in live environments
- Security sandboxes that disrupt their natural workflow
- Need for SOC2 compliance logs without changing how they work

The Paradigm Shift: Plan → Verify → Commit

The "Ghost" Sandbox

Pulse captures dangerous commands and runs them invisibly in an **isolated micro-container** first before anything touches the live host.

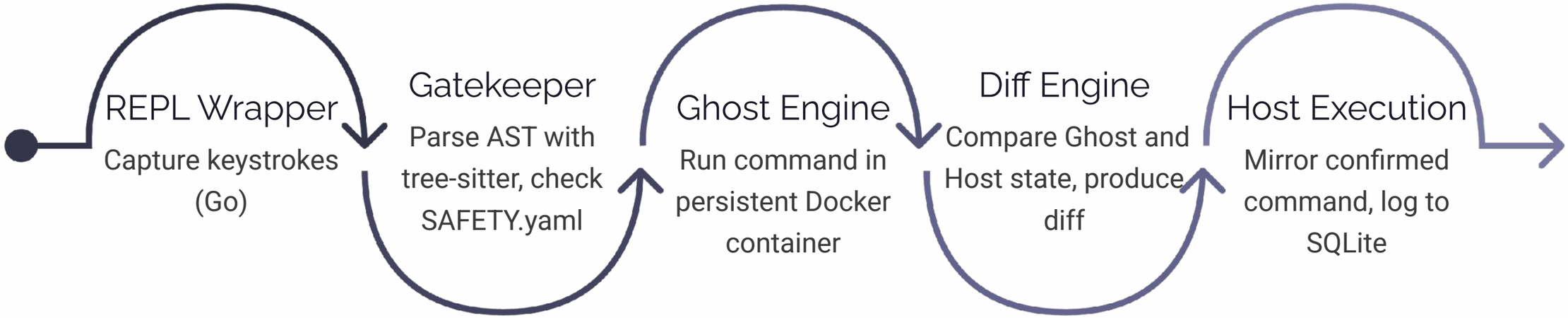
Git-Like UX for the OS

Instead of just blocking commands, Pulse shows a colourised diff (**+ added**, **- removed**) of what the command *will* do to system files — before it does it.

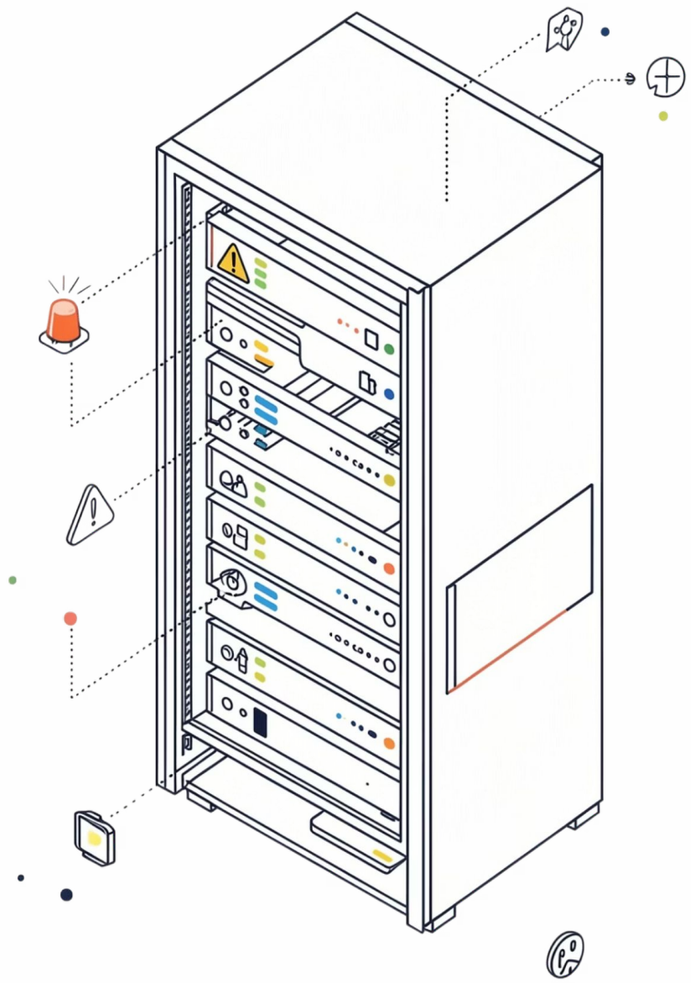
Pragmatic Speed

95% of the work is shifted to deterministic parsing and OS-level isolation, bypassing expensive AI compute to achieve **sub-50ms latency**.

The Low-Compute Architecture



The architecture is deliberately low-compute: the Gatekeeper costs zero AI compute, the Ghost Engine reuses a single long-lived paused container to avoid cold-start latency, and the Diff Engine generates a human-readable preview before any change touches the live host.



Pulse in Action: Stopping a Catastrophe

Scenario: User types `sudo rm -rf /var/log/app/old_logs/`

1

Intercept & Parse

`tree-sitter` identifies a destructive verb (`rm -rf`) targeting a monitored path. Flagged instantly.

2

Ghost Execution

Pulse runs the command instantly inside the background Ghost container. The live host is untouched.

3

Diff Generation

Pulse compares sandbox to host and outputs: *Preview: Will delete jan.log (15MB) and feb.log (12MB)*

4

Block / Commit

User is prompted: *Apply to Host? [y/N/explain]*. A $5M mistake is caught in **50 milliseconds**.

The Tech Stack

Intercept Layer

Written in **Go** for native OS bindings and single-binary compilation. Zero runtime dependencies on the target machine.

Gatekeeper

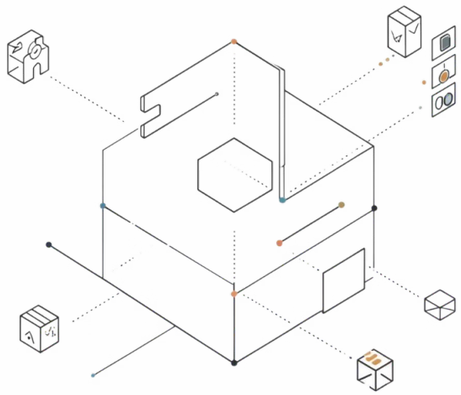
tree-sitter-bash for AST parsing (no slow regex) paired with a declarative **YAML policy engine** for extensible rule definitions.

Ghost Runtime

Standard **Docker SDK** using a persistent Alpine container. Eliminates cold-start latency entirely — the container is always warm.

High-Tier Brain

Local **Llama-3 via Ollama** or Cloud API. Invoked *only* when the user types `pulse explain` for deep script analysis.



Deployment in 2 Minutes

Judges can download, run `aegis init`, and test it instantly — no cloud account required.

Why it wins the "Working Prototype" criteria:

- No complex Kubernetes clusters or heavy cloud backends required

Shipped as a **single compiled binary** (`aegis`)

Zero Cloud Compute: Can run 100% offline

Low Resource Footprint: ~50MB RAM for the shell + ~20MB for the idle Docker ghost

Addressing the Hard Truths

We know the limitations of our own tool. Here is how Pulse handles the two most challenging edge cases judges will ask about.

🌐 Network Calls: `aws s3 rm`?

The Ghost Engine cannot sandbox external network state. Pulse Gatekeeper identifies external network binaries and, instead of a filesystem diff, forces an interactive `--confirm` prompt and tags the audit log entry as **'High Risk - External State'** — making the risk explicit rather than hiding it.

⚠️ TOCTOU (Time of Check, Time of Use)

The state of the filesystem can change between Ghost execution and Host execution. Pulse treats the Ghost as a **"best-effort heuristic preview."** For critical paths, the policy engine forces a **file-hash re-check** milliseconds prior to the final host execution, dramatically reducing the attack surface of this classic race condition.

Roadmap to Final Demo

1

Phase 1 — April 24

Architecture design, toolchain selection (Go + Docker SDK), and policy mapping. This submission.

2

Phase 2 — May 8

Build the REPL Gatekeeper (tree-sitter). Implement the persistent Docker Ghost Engine and Diff logic. Submit GitHub repo with 2-minute setup instructions.

3

Phase 3 — May 19

Implement the SQLite audit logger. Integrate optional LLM `explain` feature. Final video demo showcasing live destructive command prevention.

Why Pulse Wins

35%

Working Prototype

Highly demonstrable. We can show a broken config file being caught and a safe one passing through in real-time, live on stage.

25%

Technical Depth

Using Abstract Syntax Trees (tree-sitter) instead of regex, and persistent container management for **sub-50ms latency** — not a cloud API wrapper.

30%

UX & Business Value

Gives sysadmins the familiar UX of a Git Pull Request (Diffs) for OS operations, while automatically generating **SOC2-compliant audit logs**.